app-frontend

Student Guide

A React + Vite Client for the app-bff API Gateway

Project: app-frontend

Author: R. Seeds

Status: Verified against live project source as of 2026-08-28

SWD Java Apprenticeship — Final Assessment Companion Guide

About This Guide

This is a companion to **BFF-Setup-and-Auth-Guide.md**. That guide covers `app-bff`, the Spring Boot API gateway; this one covers `app-frontend`, the React single-page app built to consume it. If you have not stood up `app-bff` yet, do that first — `app-frontend` has nothing to talk to without it.

You are coming into this cohort with Java and Spring experience, not necessarily React. This guide does not assume you already know React — it introduces each concept (JSX, hooks, Context, client-side routing) briefly the first time it matters, right next to the code that uses it, rather than as a separate lecture up front. If you already know React, skip straight to Section 6, the anatomical guide.

Full source for every file is reproduced in Appendix A, one file per page, keyed back to the section that discusses it — the same convention used in the BFF guide.

Table of Contents

1. What This App Does

`app-frontend` is a small single-page app (SPA) with four screens:

Screen	Route	What it does
Login	/login	Authenticates against app-bff, stores the returned JWT
Register	/register	Creates a new account, then redirects to Login
Notes	/notes	Lists, creates, and deletes the logged-in user's notes; also shows the /api/greet message
Admin	/admin	Lists every registered user and their enabled status — only reachable by a user whose JWT carries the ROLE_ADMIN authority

There is no server-rendering and no build-time API calls — everything happens in the browser, after the page loads, by calling `app-bff` directly over HTTPS.

2. Technical Overview

Stack: React 19, built and served by Vite, client-side routing via `react-router-dom` v7, plain external CSS files (no CSS framework, no inline style attributes).

Architecture: one page (`index.html`) that loads a JavaScript bundle. Everything after that — what is on screen, what URL the browser shows, what data is loaded — is handled by React running in the browser. The app never talks to any server except `app-bff`, at `https://localhost:8443/api`.

Where this sits relative to app-bff: the two projects are completely separate processes, on separate ports, usually even started from separate terminals. `app-bff` does not know or care that a React app exists — it just exposes REST endpoints. `app-frontend` is one possible client of those endpoints; you could swap it out for a mobile app, a different frontend framework, or curl, and the backend would not need to change — well, almost: see the CORS note below.

Authentication flow, end to end

- User submits the Login form → POST /api/auth/login with {username, password}.
- app-bff returns {token: "<JWT>"} — a compact, RSA-signed token (see the BFF guide, Section 3, for how it is signed).
- The frontend keeps that token in memory only (a React state variable) — never in localStorage, never in a cookie. See Section 7 for why.
- Every subsequent request to a protected endpoint attaches Authorization: Bearer <token>.
- If the server ever responds 401 Unauthorized, the frontend treats that as "your session is over," clears its in-memory state, and the next protected page bounces you back to /login.

CORS — the one thing that had to change on the backend

A browser treats `http://localhost:5173` (or whatever port Vite happens to use) and `https://localhost:8443` as two different, unrelated origins, even though both say "localhost." Without explicit permission, the browser blocks the frontend's JavaScript from reading the backend's responses. `app-bff`'s `SecurityConfig.java` now has a CORS configuration that allows any localhost/127.0.0.1 origin on any port — see the BFF guide's `SecurityConfig.java` listing, and Section 7 below for why it is a wildcard rather than one fixed port.

3. React Concepts You'll See

Just enough to read the code in Section 6 — not a full React course.

Components & JSX

A React "component" is just a JavaScript function that returns markup. That markup is written in JSX — HTML-looking syntax directly inside JavaScript. `{name}` drops a JavaScript value into the markup; `<Greeting name="admin" />` elsewhere in the code renders it. That is the entire mental model — components are functions, JSX is their return value.

Hooks

Hooks are functions (always starting with `use`) that let a plain function-component have memory and side effects, which a normal JavaScript function cannot do on its own:

- **useState(initialValue)** — gives a component a piece of state that persists across re-renders, and a setter function to change it.
- **useEffect(fn, deps)** — runs fn after the component renders, and again whenever anything in the deps array changes. With deps = [], it runs once, when the component first mounts — that is how NotesPage loads notes as soon as the page appears.
- **useCallback(fn, deps) / useMemo(fn, deps)** — both "remember" a function or value across re-renders instead of recreating it every time, as long as deps has not changed. You will see these in AuthContext keeping the shared auth object stable.

Context API

Passing data from a top-level component down through five layers of intermediate components just so the bottom one can use it ("prop drilling") gets tedious fast. `createContext` plus a `<Context value={...}>` wrapper lets any descendant read that value directly via `useContext`, no matter how deeply nested. `AuthContext` is exactly this: the logged-in user, their token, and the login/logout functions, available to any page or component without passing them around manually.

React Router

- `<BrowserRouter>` — wraps the app, hooks into the browser's URL.
- `<Routes>` / `<Route path="..." element={...}>` — maps a URL path to a component.
- `<Link to="/notes">` — like an `<a>`, but navigates without a full page reload (preserving in-memory React state).
- `<Navigate to="/login" />` — programmatically redirects.
- `useNavigate()` / `useLocation()` — hooks for navigating and reading the current URL from inside a component.

4. Prerequisites

- Node.js and npm installed.
- app-bff built, its database initialized, and running on https://localhost:8443 — see BFF-Setup-and-Auth-Guide.md. The frontend has nothing to call without it.
- app-bff's SecurityConfig.java must include the CORS configuration described in Section 2. If you are working from a copy of app-bff made before that change, the frontend will load but every API call will fail silently in the browser console with a CORS error.

5. Build & Run

1. Open a terminal in app-frontend/.

2. Install dependencies (first time only):

```
npm install
```

3. Start the dev server:

```
npm run dev
```

4. Vite prints a URL, typically http://localhost:5173/ — but if that port is already busy (common if you have other projects running), Vite will pick the next free one and print whatever it actually used. That is fine — the backend's CORS rule accepts any localhost port, not just one hardcoded value.

5. Open that URL in a browser.

First-run walkthrough

- **Register** a user.
- **Log in** — you land on the Notes page, and should see "Hello, <your-username>" (that is the /api/greet call succeeding).
- **Create a note**, confirm it appears in the list, then **delete** it.
- To see the **Admin** page, you need a user whose authorities row is ROLE_ADMIN, and there is no UI path to create one — the register endpoint always assigns ROLE_USER (see the BFF guide, Section 6). Grant it directly in Postgres, then log out and back in so a fresh token carries it — the JWT's authorities are baked in at login time.

```
INSERT INTO authorities (username, authority) VALUES ('<your-username>', 'ROLE_ADMIN');
```

6. Anatomical Guide: File by File

6.1 Project Configuration

- **package.json** (A.1) — declares the three runtime dependencies (react, react-dom, react-router-dom) and the dev/build/preview scripts that npm run <script> invokes. Nothing hand-tuned here beyond what npm create vite produced, plus adding react-router-dom.
- **vite.config.js** (A.2) — Vite's own configuration. Deliberately minimal: just the React plugin (which enables JSX compilation and Fast Refresh — instant in-browser updates as you edit). No fixed port is configured, on purpose (see Section 7) — Vite is left free to pick whatever port is available.
- **index.html** (A.3) — the one real HTML page in the whole app. Its only meaningful content is (where React mounts everything) and a <script type="module" src="/src/main.jsx"> tag that loads the actual app.

6.2 Entry Point — src/main.jsx

(A.4) The first JavaScript that runs. Three things happen here, outside-in: createRoot(...).render(...) hands React control of that and tells it what to draw inside it. <BrowserRouter> turns on client-side routing for everything inside it. <StrictMode> is a development-only wrapper that helps catch bugs by intentionally double-invoking some functions; it has no effect on the production build.

6.3 src/App.jsx — Routing and Layout

(A.5) This is where the four screens from Section 1 get wired to their URLs, and it is the only place AuthProvider (Section 6.4) gets mounted — everything inside <AuthProvider> can call useAuth(). Each protected route is wrapped in <ProtectedRoute> (Section 6.6), which decides whether the requested page is actually allowed to render. App.jsx also decides when the NavBar (Section 6.7) should even appear — it is hidden on /login and /register, since there is no logged-in user yet to show a nav bar for.

6.4 src/context/AuthContext.jsx — The Auth Brain

(A.6) This is the one file every other page and component depends on, directly or indirectly. It holds three pieces of state — token, username, isAdmin — and exposes five operations: login, register, logout, and authedFetch.

decodeScope(token) — a JWT is three base64url-encoded segments joined by dots: header.payload.signature. This function grabs the middle segment, decodes it, and reads out the scope claim (the space-joined authority list TokenService put there — see the BFF guide, Section 6). It does not verify the token's signature — it cannot, since it does not have the backend's private key, and it does not need to. This decoded value is used only to decide what the UI shows (an Admin nav link, or not); the server enforces the real security check independently, on every request, regardless of what this function returns. See Section 7 for why it checks for the bare string ROLE_ADMIN rather than SCOPE_ROLE_ADMIN.

authedFetch — every page that needs to call a protected endpoint uses this instead of calling apiFetch (Section 6.5) directly. It attaches the current token automatically, and if the response comes back 401, it calls logout() before re-throwing the error — so any page using it gets "kick back to login on an expired session" for free, without repeating that logic itself.

Why useCallback and useMemo are all over this file: every function and the final value object are wrapped in one of these two hooks. Without that, a fresh copy of every function would be created on every single re-render, and anything depending on the context (like authedFetch's dependency array in NotesPage) would think its dependencies changed constantly, causing wasted re-fetches. Wrapping them keeps the same function reference across renders unless something it actually depends on changes.

6.5 src/api/client.js — Talking to the Backend

(A.7) The lowest-level piece: one function, apiFetch, wrapping the browser's built-in fetch. It always targets https://localhost:8443/api, always sends Content-Type: application/json, adds Authorization: Bearer <token> when a token is passed in, and — importantly — throws a custom ApiError (carrying the HTTP status code) on any non-2xx response, instead of fetch's default behavior of only rejecting on network failure. That is what lets LoginPage distinguish "wrong password" (401) from "backend isn't running" (network error, no status at all), and lets RegisterPage distinguish "username taken" (409) from anything else.

6.6 src/components/ProtectedRoute.jsx — The Gatekeeper

(A.8) A small wrapper component used in App.jsx's route definitions. Its logic is two if statements: no token means bounce to Login. Route requires admin and this user is not one means bounce to Notes instead of showing a broken/empty admin page. Otherwise, render whatever page it was wrapping. This is a **UI convenience, not a security boundary** — a user could, in principle, tamper with the running JavaScript to skip this check entirely. The actual security boundary is SecurityConfig.java's hasAuthority("SCOPE_ROLE_ADMIN") rule on the backend, which this component's requireAdmin check is simply trying to match so the UI does not show a page that would just fail anyway.

6.7 src/components/NavBar.jsx + NavBar.css

(A.9, A.10) The top bar shown on every authenticated page. Reads username and isAdmin from useAuth() to decide what to display — the Admin link only renders when {isAdmin && <Link to="/admin">Admin</Link>}, which is JSX shorthand for "render this only if the condition is true." The logout button calls logout() (clearing the in-memory auth state) and then navigate('/login').

6.8 src/pages/LoginPage.jsx & RegisterPage.jsx (+ AuthForm.css)

(A.11, A.12, A.13) Both are plain controlled forms: each input's value is held in a useState variable and updated on every keystroke via onChange, rather than letting the browser manage the input's value itself. This is the standard React pattern — it means the component's state is always the single source of truth for what is in the box.

RegisterPage navigates to /login on success, passing {state: {registered: true}} — LoginPage reads that via useLocation().state?.registered to show a one-time "Account created" message, without that flag ever touching the URL itself (no ?registered=true query string). Both share AuthForm.css for styling, since they are visually identical card-centered forms.

6.9 src/pages/NotesPage.jsx + NotesPage.css

(A.14, A.15) The main authenticated screen. On mount (useEffect with an empty dependency array), it fires two calls through authedFetch: one to /greet, one to /notes. Create and delete both call the backend, then update local state directly from the response rather than re-fetching the whole list — for example setNotes((current) => [...current, created]) appends the newly created note returned by the POST /api/notes call.

6.10 src/pages/AdminPage.jsx + AdminPage.css

(A.16, A.17) The simplest data page: one useEffect on mount, one authedFetch('/admin/users') call, rendered as a table. Note the field name it reads off each user: user.userName (capital N), matching the UserSummary record's userName field in AppController.java (BFF guide, Appendix A.12) — not user.username. This page only ever renders for a user whose token carries ROLE_ADMIN (enforced by ProtectedRoute), but even if it somehow rendered for a non-admin, the backend's own hasAuthority("SCOPE_ROLE_ADMIN") check would still reject the request with a 403 before any data reached the browser.

6.11 src/index.css — Global Styles

(A.18) The only global stylesheet: a box-sizing: border-box reset and the page's base font/background/text color. Every other visual style lives in a CSS file scoped to the component or page it belongs to (NavBar.css, AuthForm.css, NotesPage.css, AdminPage.css) — no inline style attributes anywhere in the app.

7. Design Decisions Worth Understanding

Token kept in memory only, not localStorage

A React state variable disappears on page refresh — logging back in after every reload is a real cost. The upside: localStorage is readable by any JavaScript running on the page, including an injected script from an XSS vulnerability elsewhere in the app; an in-memory value in AuthContext is not reachable that way. For this app, that tradeoff was chosen deliberately in favor of the more secure option.

Why decodeScope checks for ROLE_ADMIN, not SCOPE_ROLE_ADMIN

TokenService (BFF guide, Appendix A.7) puts the raw authority string from the database — ROLE_ADMIN — into the JWT's scope claim. Spring Security's default JwtAuthenticationConverter adds the SCOPE_ prefix on the server, when it turns an incoming token into an Authentication object for that one request — that prefixing never happens to the token itself, and never reaches the browser. So the frontend, reading the token directly, sees ROLE_ADMIN; the backend's authorization rule, reading its own internally-converted representation, checks for SCOPE_ROLE_ADMIN. Same underlying fact, two different string forms, because one is the raw claim and the other is Spring's server-side convention for representing it.

CORS: any localhost port, not one fixed port

The first working version of this app pinned Vite to a specific port and hardcoded that exact origin into the backend's CORS allow-list. That breaks the moment two people's dev environments differ even slightly — on one machine, Vite's default port 5173 was already taken by an unrelated process, so it picked something else entirely, silently invalidating a hardcoded value. SecurityConfig.java's CORS bean now uses setAllowedOriginPatterns(List.of("http://localhost:*", "http://127.0.0.1:*")) instead — a pattern, not an exact match — so it works no matter which port Vite ends up on for any given student. This is safe specifically because this app never sends cookies cross-origin (the JWT goes in a manually-set header, not a cookie), so the usual restriction against combining wildcard origins with credentialed requests does not apply here.

No refresh token, and why that is fine here

There is no /api/auth/refresh endpoint in app-bff — a token is valid for exactly one hour, or until the backend restarts (its signing key is regenerated fresh every boot; see BFF guide, Section 3). authedFetch's blanket "401 to logout" behavior means the frontend does not need to guess when a token might be stale — it finds out for certain the next time it makes a request, and reacts uniformly regardless of why the token stopped working (expiry, restart, or anything else).

8. Troubleshooting

Symptom	Root Cause	Fix
Console shows a CORS error on every request	app-bff's SecurityConfig.java doesn't have the CORS bean, or the backend wasn't rebuilt/restarted after it was added	Confirm the corsConfigurationSource bean exists, mvn clean compile, restart AppBffApplication
Login form submits but shows "Login failed. Is the backend running?"	app-bff isn't running, or isn't reachable at https://localhost:8443	Start (or restart) the backend; confirm with https://localhost:8443/api/greet in a browser tab (expect a 401, not a connection error)
'useAuth must be used within an AuthProvider' thrown in the console	Some component calling useAuth() is rendered outside <AuthProvider> in App.jsx	Make sure any component reading auth state is nested inside <AuthProvider><Layout /></AuthProvider>
Page reload logs you out unexpectedly	This is expected — the token lives only in React state (Section 7), which is wiped on any full page reload	Log back in; remember this is a deliberate tradeoff, not a bug
A user has ROLE_ADMIN in the authorities table but still can't see /admin	Their current JWT was issued before the database change — authorities are baked into the token at login time	Log out and log back in to get a fresh token
Admin/Notes page suddenly errors out or shows nothing after restarting the backend	app-bff regenerates its RSA signing keypair on every restart, instantly invalidating every previously-issued token	Log out and log back in
npm run dev prints a port other than 5173	Something else on the machine is already using 5173 (and possibly 5174); Vite falls back automatically	No action needed — CORS is configured to accept any localhost port

Appendix A: Full Source Listings

Entry	File	Discussed in
A.1	app-frontend/package.json	Section 6.1
A.2	app-frontend/vite.config.js	Section 6.1
A.3	app-frontend/index.html	Section 6.1
A.4	app-frontend/src/main.jsx	Section 6.2
A.5	app-frontend/src/App.jsx	Section 6.3
A.6	app-frontend/src/context/AuthContext.jsx	Section 6.4
A.7	app-frontend/src/api/client.js	Section 6.5
A.8	app-frontend/src/components/ProtectedRoute.jsx	Section 6.6
A.9	app-frontend/src/components/NavBar.jsx	Section 6.7
A.10	app-frontend/src/components/NavBar.css	Section 6.7
A.11	app-frontend/src/pages/LoginPage.jsx	Section 6.8
A.12	app-frontend/src/pages/RegisterPage.jsx	Section 6.8
A.13	app-frontend/src/pages/AuthForm.css	Section 6.8
A.14	app-frontend/src/pages/NotesPage.jsx	Section 6.9
A.15	app-frontend/src/pages/NotesPage.css	Section 6.9
A.16	app-frontend/src/pages/AdminPage.jsx	Section 6.10
A.17	app-frontend/src/pages/AdminPage.css	Section 6.10
A.18	app-frontend/src/index.css	Section 6.11

A.1 package.json

> *Discussed in Section 6.1.*

```
{
  "name": "app-frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.3"
  },
  "devDependencies": {
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.4",
    "@vitejs/plugin-react": "^6.1.0",
    "oxlint": "^1.79.0",
    "vite": "^8.2.2"
  }
}
```

A.2 vite.config.js

> *Discussed in Section 6.1.*

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

A.3 index.html

> *Discussed in Section 6.1.*

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>app-frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

A.4 src/main.jsx

> *Discussed in Section 6.2.*

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

A.5 src/App.jsx

> *Discussed in Section 6.3.*

```
import { Navigate, Route, Routes, useLocation } from 'react-router-dom'
import { AuthProvider, useAuth } from './context/AuthContext'
import { ProtectedRoute } from './components/ProtectedRoute'
import { NavBar } from './components/NavBar'
import { LoginPage } from './pages/LoginPage'
import { RegisterPage } from './pages/RegisterPage'
import { NotesPage } from './pages/NotesPage'
import { AdminPage } from './pages/AdminPage'

function Layout() {
  const { token } = useAuth()
  const location = useLocation()
  const showNav = token && location.pathname !== '/login' && location.pathname !== '/register'

  return (
    <>
      {showNav && <NavBar />}
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route
          path="/notes"
          element={
            <ProtectedRoute>
              <NotesPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/admin"
          element={
            <ProtectedRoute requireAdmin>
              <AdminPage />
            </ProtectedRoute>
          }
        />
        <Route path="*" element={<Navigate to="/notes" replace />} />
      </Routes>
    </>
  )
}

function App() {
  return (
    <AuthProvider>
      <Layout />
    </AuthProvider>
  )
}

export default App
```

A.6 src/context/AuthContext.jsx

> *Discussed in Section 6.4.*

```
import { createContext, useCallback, useContext, useMemo, useState } from 'react'
import { apiFetch, ApiError } from '../api/client'

const AuthContext = createContext(null)

function decodeScope(token) {
  try {
    const payload = token.split('.')[1]
    const normalized = payload.replace(/-/g, '+').replace(/_/g, '/')
    const json = atob(normalized)
    const claims = JSON.parse(json)
    return typeof claims.scope === 'string' ? claims.scope.split(' ') : []
  } catch {
    return []
  }
}

export function AuthProvider({ children }) {
  const [token, setToken] = useState(null)
  const [username, setUsername] = useState(null)
  const [isAdmin, setIsAdmin] = useState(false)

  const login = useCallback(async (loginUsername, password) => {
    const data = await apiFetch('/auth/login', {
      method: 'POST',
      body: { username: loginUsername, password },
    })
    const authorities = decodeScope(data.token)
    setToken(data.token)
    setUsername(loginUsername)
    setIsAdmin(authorities.includes('ROLE_ADMIN'))
  }, [])

  const register = useCallback(async (registerUsername, password) => {
    await apiFetch('/auth/register', {
      method: 'POST',
      body: { username: registerUsername, password },
    })
  }, [])

  const logout = useCallback(() => {
    setToken(null)
    setUsername(null)
    setIsAdmin(false)
  }, [])

  const authedFetch = useCallback(
    async (path, options = {}) => {
      try {
        return await apiFetch(path, { ...options, token })
      } catch (err) {
        if (err instanceof ApiError && err.status === 401) {
          logout()
        }
```

```
        throw err
      }
    },
    [token, logout],
  )

  const value = useMemo(
    () => ({ token, username, isAdmin, login, register, logout, authedFetch }),
    [token, username, isAdmin, login, register, logout, authedFetch],
  )

  return <AuthContext value={value}>{children}</AuthContext>
}

export function useAuth() {
  const context = useContext(AuthContext)
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider')
  }
  return context
}
```

A.7 src/api/client.js

> *Discussed in Section 6.5.*

```
const BASE_URL = 'https://localhost:8443/api'

export class ApiError extends Error {
  constructor(status, message) {
    super(message)
    this.status = status
  }
}

export async function apiFetch(path, { method = 'GET', body, token } = {}) {
  const headers = { 'Content-Type': 'application/json' }
  if (token) {
    headers.Authorization = `Bearer ${token}`
  }

  const response = await fetch(`${BASE_URL}${path}`, {
    method,
    headers,
    body: body !== undefined ? JSON.stringify(body) : undefined,
  })

  if (!response.ok) {
    const text = await response.text().catch(() => '')
    throw new ApiError(response.status, text || response.statusText)
  }

  if (response.status === 204) {
    return null
  }

  const contentType = response.headers.get('content-type') || ''
  if (contentType.includes('application/json')) {
    return response.json()
  }
  return response.text()
}
```

A.8 src/components/ProtectedRoute.jsx

> *Discussed in Section 6.6.*

```
import { Navigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

export function ProtectedRoute({ children, requireAdmin = false }) {
  const { token, isAdmin } = useAuth()

  if (!token) {
    return <Navigate to="/login" replace />
  }

  if (requireAdmin && !isAdmin) {
    return <Navigate to="/notes" replace />
  }

  return children
}
```

A.9 src/components/NavBar.jsx

> *Discussed in Section 6.7.*

```
import { Link, useNavigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
import './NavBar.css'

export function NavBar() {
  const { username, isAdmin, logout } = useAuth()
  const navigate = useNavigate()

  function handleLogout() {
    logout()
    navigate('/login')
  }

  return (
    <nav className="navbar">
      <div className="navbar-links">
        <Link to="/notes">Notes</Link>
        {isAdmin && <Link to="/admin">Admin</Link>}
      </div>
      <div className="navbar-user">
        <span className="navbar-username">{username}</span>
        <button type="button" className="navbar-logout" onClick={handleLogout}>
          Log out
        </button>
      </div>
    </nav>
  )
}
```

A.10 src/components/NavBar.css

> *Discussed in Section 6.7.*

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1rem 1.5rem;
  background-color: #1f2933;
  color: #f8fafc;
}

.navbar-links {
  display: flex;
  align-items: center;
  gap: 1.25rem;
}

.navbar-links a {
  color: #cbd2d9;
  text-decoration: none;
  font-weight: 500;
}

.navbar-links a:hover {
  color: #f8fafc;
}

.navbar-user {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.navbar-username {
  font-weight: 600;
}

.navbar-logout {
  background-color: #ef4444;
  color: #f8fafc;
  border: none;
  border-radius: 4px;
  padding: 0.4rem 0.9rem;
  cursor: pointer;
  font-weight: 500;
}

.navbar-logout:hover {
  background-color: #dc2626;
}
```

A.11 src/pages/LoginPage.jsx

> *Discussed in Section 6.8.*

```
import { useState } from 'react'
import { Link, useLocation, useNavigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
import { ApiError } from '../api/client'
import './AuthForm.css'

export function LoginPage() {
  const { login } = useAuth()
  const navigate = useNavigate()
  const location = useLocation()
  const [username, setUsername] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState(null)
  const [submitting, setSubmitting] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    setError(null)
    setSubmitting(true)
    try {
      await login(username, password)
      navigate('/notes')
    } catch (err) {
      if (err instanceof ApiError && err.status === 401) {
        setError('Invalid username or password.')
      } else {
        setError('Login failed. Is the backend running?')
      }
    } finally {
      setSubmitting(false)
    }
  }

  return (
    <div className="auth-page">
      <div className="auth-card">
        <h1>Log in</h1>
        {location.state?.registered && (
          <p className="auth-success">Account created — log in below.</p>
        )}
        {error && <p className="auth-error">{error}</p>}
        <form onSubmit={handleSubmit}>
          <div className="auth-field">
            <label htmlFor="username">Username</label>
            <input
              id="username"
              value={username}
              onChange={(event) => setUsername(event.target.value)}
              autoComplete="username"
              required
            />
          </div>
          <div className="auth-field">
            <label htmlFor="password">Password</label>
            <input
```

```
              id="password"
              type="password"
              value={password}
              onChange={(event) => setPassword(event.target.value)}
              autoComplete="current-password"
              required
            />
          </div>
          <button type="submit" className="auth-submit" disabled={submitting}>
            {submitting ? 'Logging in…' : 'Log in'}
          </button>
        </form>
        <p className="auth-switch">
          Need an account? <Link to="/register">Register</Link>
        </p>
      </div>
    </div>
  )
}
```

A.12 src/pages/RegisterPage.jsx

> *Discussed in Section 6.8.*

```
import { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
import { ApiError } from '../api/client'
import './AuthForm.css'

export function RegisterPage() {
  const { register } = useAuth()
  const navigate = useNavigate()
  const [username, setUsername] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState(null)
  const [submitting, setSubmitting] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    setError(null)
    setSubmitting(true)
    try {
      await register(username, password)
      navigate('/login', { state: { registered: true } })
    } catch (err) {
      if (err instanceof ApiError && err.status === 409) {
        setError('That username is already taken.')
      } else {
        setError('Registration failed. Is the backend running?')
      }
    } finally {
      setSubmitting(false)
    }
  }

  return (
    <div className="auth-page">
      <div className="auth-card">
        <h1>Create an account</h1>
        {error && <p className="auth-error">{error}</p>}
        <form onSubmit={handleSubmit}>
          <div className="auth-field">
            <label htmlFor="username">Username</label>
            <input
              id="username"
              value={username}
              onChange={(event) => setUsername(event.target.value)}
              autoComplete="username"
              required
            />
          </div>
          <div className="auth-field">
            <label htmlFor="password">Password</label>
            <input
              id="password"
              type="password"
              value={password}
              onChange={(event) => setPassword(event.target.value)}
```

```
              autoComplete="new-password"
              required
            />
          </div>
          <button type="submit" className="auth-submit" disabled={submitting}>
            {submitting ? 'Creating account…' : 'Register'}
          </button>
        </form>
        <p className="auth-switch">
          Already have an account? <Link to="/login">Log in</Link>
        </p>
      </div>
    </div>
  )
}
```

A.13 src/pages/AuthForm.css

> *Discussed in Section 6.8.*

```
.auth-page {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f0f4f8;
}

.auth-card {
  background-color: #ffffff;
  border-radius: 8px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.15);
  padding: 2rem;
  width: 320px;
}

.auth-card h1 {
  margin-bottom: 1.5rem;
  font-size: 1.4rem;
  text-align: center;
  color: #1f2933;
}

.auth-field {
  display: flex;
  flex-direction: column;
  margin-bottom: 1rem;
}

.auth-field label {
  font-size: 0.85rem;
  font-weight: 600;
  color: #3e4c59;
  margin-bottom: 0.35rem;
}

.auth-field input {
  padding: 0.55rem 0.7rem;
  border: 1px solid #cbd2d9;
  border-radius: 4px;
  font-size: 1rem;
}

.auth-submit {
  width: 100%;
  padding: 0.6rem;
  margin-top: 0.5rem;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
}
```

```
.auth-submit:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.auth-submit:hover:not(:disabled) {
  background-color: #1d4ed8;
}

.auth-error {
  color: #dc2626;
  font-size: 0.85rem;
  margin-bottom: 1rem;
}

.auth-success {
  color: #15803d;
  font-size: 0.85rem;
  margin-bottom: 1rem;
}

.auth-switch {
  margin-top: 1.25rem;
  text-align: center;
  font-size: 0.85rem;
  color: #3e4c59;
}

.auth-switch a {
  color: #2563eb;
  font-weight: 600;
  text-decoration: none;
}
```

A.14 src/pages/NotesPage.jsx

> *Discussed in Section 6.9.*

```
import { useEffect, useState } from 'react'
import { useAuth } from '../context/AuthContext'
import './NotesPage.css'

export function NotesPage() {
  const { username, authedFetch } = useAuth()
  const [greeting, setGreeting] = useState('')
  const [notes, setNotes] = useState([])
  const [title, setTitle] = useState('')
  const [content, setContent] = useState('')
  const [error, setError] = useState(null)
  const [submitting, setSubmitting] = useState(false)

  useEffect(() => {
    authedFetch('/greet').then(setGreeting).catch(() => {})
    loadNotes()
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [])

  async function loadNotes() {
    try {
      const data = await authedFetch('/notes')
      setNotes(data)
    } catch {
      setError('Could not load notes.')
    }
  }

  async function handleCreate(event) {
    event.preventDefault()
    setError(null)
    setSubmitting(true)
    try {
      const created = await authedFetch('/notes', {
        method: 'POST',
        body: { title, content },
      })
      setNotes((current) => [...current, created])
      setTitle('')
      setContent('')
    } catch {
      setError('Could not create note.')
    } finally {
      setSubmitting(false)
    }
  }

  async function handleDelete(id) {
    try {
      await authedFetch(`/notes/${id}`, { method: 'DELETE' })
      setNotes((current) => current.filter((note) => note.id !== id))
    } catch {
      setError('Could not delete note.')
    }
  }
```

```
  return (
    <div className="notes-page">
      <p className="notes-greeting">{greeting || `Hello, ${username}`}</p>
      {error && <p className="notes-error">{error}</p>}

      <form className="notes-form" onSubmit={handleCreate}>
        <input
          placeholder="Title"
          value={title}
          onChange={(event) => setTitle(event.target.value)}
          required
        />
        <textarea
          placeholder="What's on your mind?"
          value={content}
          onChange={(event) => setContent(event.target.value)}
          required
        />
        <button type="submit" disabled={submitting}>
          {submitting ? 'Saving…' : 'Add note'}
        </button>
      </form>

      <div className="notes-list">
        {notes.length === 0 && <p className="notes-empty">No notes yet.</p>}
        {notes.map((note) => (
          <div className="note-card" key={note.id}>
            <h3>{note.title}</h3>
            <p>{note.content}</p>
            <button type="button" className="note-delete" onClick={() => handleDelete(note.id)}>
              Delete
            </button>
          </div>
        ))}
      </div>
    </div>
  )
}
```

A.15 src/pages/NotesPage.css

> *Discussed in Section 6.9.*

```
.notes-page {
  max-width: 640px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}

.notes-greeting {
  color: #3e4c59;
  margin-bottom: 1.5rem;
}

.notes-error {
  color: #dc2626;
  margin-bottom: 1rem;
}

.notes-form {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  background-color: #ffffff;
  border: 1px solid #e4e7eb;
  border-radius: 8px;
  padding: 1.25rem;
  margin-bottom: 2rem;
}

.notes-form input,
.notes-form textarea {
  padding: 0.55rem 0.7rem;
  border: 1px solid #cbd2d9;
  border-radius: 4px;
  font-size: 1rem;
  font-family: inherit;
}

.notes-form textarea {
  min-height: 80px;
  resize: vertical;
}

.notes-form button {
  align-self: flex-start;
  padding: 0.5rem 1.1rem;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  font-weight: 600;
  cursor: pointer;
}

.notes-form button:disabled {
  opacity: 0.6;
```

```
  cursor: not-allowed;
}

.notes-list {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.notes-empty {
  color: #7b8794;
}

.note-card {
  background-color: #ffffff;
  border: 1px solid #e4e7eb;
  border-radius: 8px;
  padding: 1rem 1.25rem;
}

.note-card h3 {
  margin-bottom: 0.4rem;
  color: #1f2933;
}

.note-card p {
  color: #3e4c59;
  white-space: pre-wrap;
  margin-bottom: 0.75rem;
}

.note-delete {
  background-color: transparent;
  color: #dc2626;
  border: 1px solid #dc2626;
  border-radius: 4px;
  padding: 0.3rem 0.7rem;
  font-size: 0.85rem;
  cursor: pointer;
}

.note-delete:hover {
  background-color: #fef2f2;
}
```

A.16 src/pages/AdminPage.jsx

> *Discussed in Section 6.10.*

```
import { useEffect, useState } from 'react'
import { useAuth } from '../context/AuthContext'
import './AdminPage.css'

export function AdminPage() {
  const { authedFetch } = useAuth()
  const [users, setUsers] = useState([])
  const [error, setError] = useState(null)

  useEffect(() => {
    authedFetch('/admin/users')
      .then(setUsers)
      .catch(() => setError('Could not load users.'))
  }, [authedFetch])

  return (
    <div className="admin-page">
      <h1>Registered Users</h1>
      {error && <p className="admin-error">{error}</p>}
      <table className="admin-table">
        <thead>
          <tr>
            <th>Username</th>
            <th>Status</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.userName}>
              <td>{user.userName}</td>
              <td className={user.enabled ? 'admin-status-enabled' : 'admin-status-disabled'}>
                {user.enabled ? 'Enabled' : 'Disabled'}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )
}
```

A.17 src/pages/AdminPage.css

> *Discussed in Section 6.10.*

```
.admin-page {
  max-width: 640px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}

.admin-page h1 {
  margin-bottom: 1.25rem;
  color: #1f2933;
}

.admin-error {
  color: #dc2626;
  margin-bottom: 1rem;
}

.admin-table {
  width: 100%;
  border-collapse: collapse;
  background-color: #ffffff;
  border: 1px solid #e4e7eb;
  border-radius: 8px;
  overflow: hidden;
}

.admin-table th,
.admin-table td {
  text-align: left;
  padding: 0.65rem 1rem;
  border-bottom: 1px solid #e4e7eb;
}

.admin-table th {
  background-color: #f0f4f8;
  font-size: 0.85rem;
  text-transform: uppercase;
  color: #3e4c59;
}

.admin-status-enabled {
  color: #15803d;
  font-weight: 600;
}

.admin-status-disabled {
  color: #dc2626;
  font-weight: 600;
}
```

A.18 src/index.css

> *Discussed in Section 6.11.*

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  background-color: #f0f4f8;
  color: #1f2933;
}
```